

Software Atelier 4: Software Engineering Project (Spring 2026)

Instructor: Dr. Andrea Mocci

TAs: Valerie Burgener, Francesco De Vito, Masoud Jamshidiyantehrani, Sasha
Toscano

Version: 1.0 (March 3, 2026)



Preamble

This document is not final, and it is not perfect. It is part of the project to go through it and find areas where there might be need of clarifications (ambiguities, underspecified aspects, etc.) or where there are inconsistencies. The instructor is the responsible person to clarify those aspects, if needed. An extremely important part of the project is to go through this document with the rest of your team and find possible issues or aspects that you think require clarifications.

1 Domain Description

Blockhop (codename) is a community game where gamers can create platformer levels, share them with other users in the system, and thus let other users play them. In particular, users compete to solve the hardest levels. This part of the document describes the domain - not all concepts might be strictly related to the user stories of this batch, it is your duty to understand what is important and what it is not.

1.1 Basic Lore

The main character is an alien arrived on the earth - and while its full objective is still not known, for this batch its objectives will be just completing some levels.

1.2 Level

A level is a self-contained space where the player navigates a character across platforms, items, obstacles, and enemies. Figure 1 shows a possible level and how it will look like on the engine.



Figure 1: A messy level and how it looks on the engine.

A level is composed of ground/blocks and game objects, each one positioned in a specific tile of the level. A level is always 14 tiles high and it has a length of 256 tiles.

1.3 Ground and Blocks

Ground and blocks are elements that cannot be destroyed. They can be terrain, like in Figure 2a, or other kind of solid blocks, like in Figure 2b.

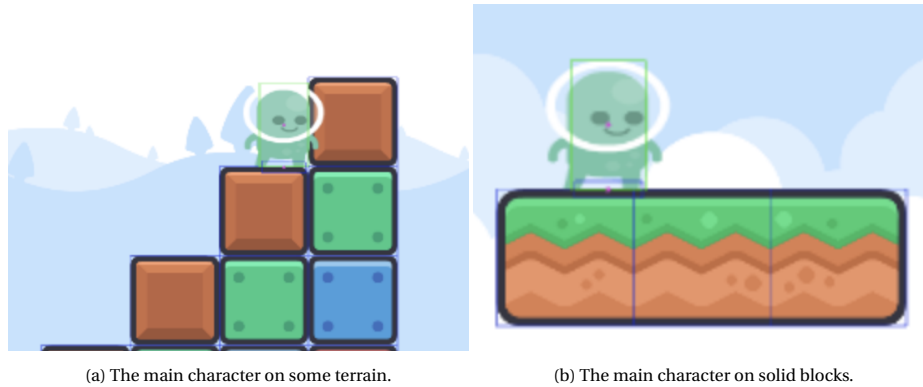


Figure 2: Examples of ground and blocks.

1.4 Items

The level starts at the green flag. The green flag is an item that can be present only *once* in a level.

Some items, like signs and the grass, are just decorations and have no effect on the gameplay other than improve the aesthetics, as shown in Figure 3.

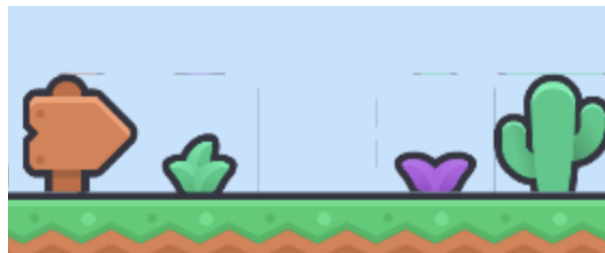


Figure 3: Examples of decoration items.

Other items are boxes, which can be destroyed, and collectibles, which can be collected. For now there is only one type of collectible, the coin, as shown in Figure 4.



Figure 4: Boxes and coins.

Coins have a value: There are gold coins (with value equal to 100), silver coins (value 25), and bronze coins (value 5).

To complete a level, the player needs to reach the exit door, which needs to be open to complete the level, as shown in Figure 5. The level can have only one exit door.

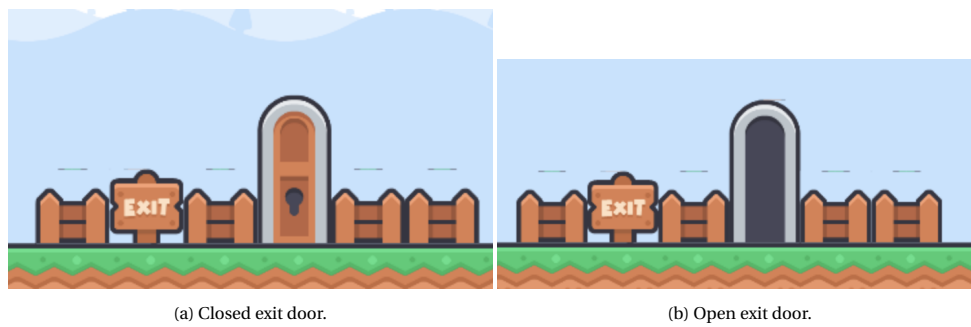


Figure 5: The exit door in its closed and open states.

1.5 Enemies

There are two kind of enemies in a level: slimes and snails, as shown in Figure 6.

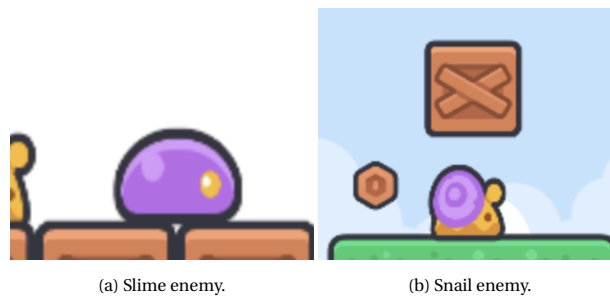


Figure 6: The two enemy types.

Slimes can be killed basically by the player jumping on top of them. A snail cannot be killed in the same way: it will hide into its shell for 5 seconds, before coming back active. A snail never goes over the end of a platform, but it goes back and forth edges. A slime instead can go over platforms, possibly falling down. When interacting with boxes, grounds, or other enemies, enemies turn into the opposite direction.

An enemy may damage the player. A second damage of the player kills it, and the level must be restarted.

1.5.1 Shells

A shell can be thrown by the player by pushing it into some direction. A thrown shell may kill enemies and destruct boxes, but it will bounce back ground and blocks. Furthermore, if it hits the player, it will damage it. The player can stop the shell by jumping on it.

1.6 Clear Conditions

A level may have a clear condition. If it has no clear condition, the exit door is open at the beginning of the level. Instead, a clear condition, when triggered, it will determine the opening of the exit door. A clear condition, once reached, cannot be lost (for this batch at least).

A clear condition can be destroying a certain number of boxes, collecting a certain amount of coin value, or killing a certain amount of enemy types (i.e., slimes or snails).

2 (Some) Software Requirements

The engine will be provided in two weeks after Lab 03, where we will start the frontend, and this document will be updated. For now, what you have to do is to concentrate on understanding the domain, modeling it, and constructing the backend. The engine will use a standard format for levels - in the end, it will require basically some json format. The basic format of this file looks like this:

```
{
  "backgroundcolor": "#000000",
  "compressionlevel": -1,
  "height": 14,
  "width": 256,
  "infinite": false,
  "layers": [
    { "...": "..." },
    { "...": "..." }
  ],
  "nextlayerid": 5,
  "nextobjectid": 116,
  "orientation": "orthogonal",
  "renderorder": "right-down",
  "tiledversion": "1.11.2",
  "tileheight": 128,
  "tilesets": [
    { "...": "..." }
  ],
  "tilewidth": 128,
  "type": "map",
  "version": "1.10",
}
```

All fields must be pretty much constant here, except for the *layers*. The layers are what actually specify a level, and they are based on the concept of a *tile*, each one specified via a *tileset*.

There is only one tileset in our case. A **tileset** is a catalogue that assigns a numeric **id** to every visual element in the game — ground tiles, blocks, items, enemies, and so on. The tileset is **fixed** for this batch and will be provided; you do not need to create or modify it. Each entry in the tileset has a unique id starting from 0, as illustrated below:

The ids are important because they are the values your backend will store and expose: when the engine renders a level it looks up each id in the tileset to find the corresponding image. The JSON representation of the tileset looks like the following (only the first two tiles are shown for brevity):

```
{
  "name": "blockhop-tileset",
  "firstgid": 1,
  "tilecount": 42,
  "tilewidth": 128,
  "tileheight": 128,
  "tiles": [
    { "id": 0, "...": "..." },
    { "id": 1, "...": "..." }
  ]
}
```

When a tile is placed in a level, it is referenced by its **gid**. Your backend should know and there therefore store somewhere tile gids (integers) to represent what occupies each position (tile) in a level. The gid of a tile is given by the **firstgid** field of the tileset plus the **id** of a tile. The ids in the tileset are assigned left-to-right, top-to-bottom. Considering Figure 7, the top-left tile has **id** == 0 and **gid** == 1, while the first one in the second row has (in this case) **id** == 10 and **gid** == 11. The tileset is 10 tiles-wide. Each tile has also a **type** field that identifies the type of tile.

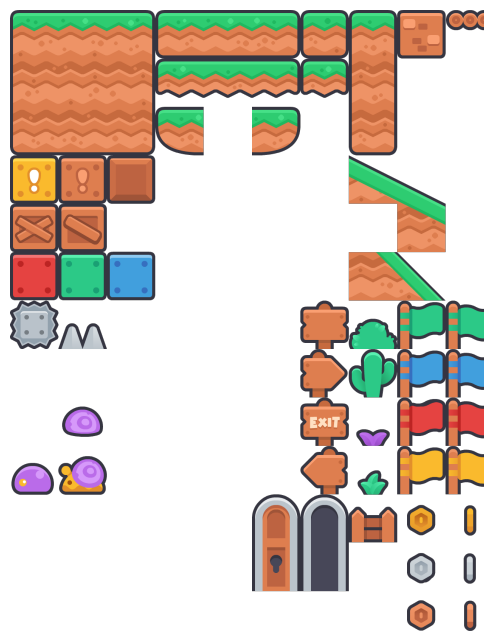


Figure 7: The tileset image — each cell corresponds to one tile id.

Tiles contain all game elements excluding the player, i.e. enemies, items, ground elements. However, the ground elements and the other objects are represented in two different *layers*. Furthermore, certain elements should ever happen only in their specific *layer*.

2.1 First Layer: World Layer

The first layer, or world layer, is composed only of tiles of type **Ground**. These are terrains and blocks. Its JSON representation is a tile layer, where each cell of the level grid is encoded as a gid (0 means empty):

```
{  
  "data": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
    0, 0, 0, 0, 4, 5, 6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
    4, 6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 14, 15, 16, 0, 0, 0, 0, 0, 7, 0, 0, 0, 14, 15, 15, 16,  
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 13,  
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 13,  
    1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 0, 0, 0, 0, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 13,  
    11, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 13, 0, 0, 0, 0, 11, 12, 12, 12, 12, 12, 13,  
    11, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 13, 0, 0, 0, 0, 11, 12, 12, 12, 12, 12, 13],  
  "height": 14,  
  "id": 1,  
  "name": "World",  
  "opacity": 1,  
  "type": "tilelayer",  
  "visible": true,  
  "width": 256,  
  "x": 0,  
  "y": 0
```

The data array contains `width × height` gid values, listed row by row from the top-left to the bottom-right.

A value of 0 means the cell is empty; any other value is the gid of the ground tile placed there.

2.2 Object Layer

The second layer is structured differently and is composed of a list of objects, and includes specifications for the positions and properties of things like decorations, items, enemies, etc. For example:

```
{
  "draworder": "topdown",
  "id": 2,
  "name": "QMLayer",
  "opacity": 1,
  "type": "objectgroup",
  "visible": true,
  "x": 0,
  "y": 0
  "objects": [ "... " ]
}
```

The following object specification identifies a box (with **gid == 41**):

```
{
  "gid": 41,
  "id": 44,
  "height": 128,
  "width": 128,
  "name": "",
  "rotation": 0,
  "type": "",
  "visible": true,
  "x": 2432,
  "y": 1152
}
```

This is the corresponding tile spec in the tileset:

```
{
  "id": 40,
  "type": "BoxDouble"
}
```

For all objects, height and width are fixed (128), while the positions are in absolute pixels and they are always multiples of 128, identifying the top-right of the object.

The following tile specs instead identify some decoration and enemies:

```
{
  "id": 87,
  "type": "Decoration"
},
{
  "id": 90,
  "type": "Enemy_Slime_Normal"
},
{
  "id": 91,
  "type": "Enemy_Snail"
}
```

This is a possible corresponding object spec:

```
{
  "gid":91,
  "height":128,
  "id":97,
  "name":"",
  "rotation":0,
  "type":"",
  "visible":true,
  "width":128,
  "x":5760,
  "y":1024
}
```

In the object layer, ids must be unique.

One important aspect is that for the exit door, you do not need to put in the object layer the corresponding top tile - the engine will do that automatically.

Another important aspect is that objects can have properties. The only relevant property for this batch is that boxes can have a content:

```
{
  "gid":41,
  "height":128,
  "id":101,
  "name":"",
  "properties":[
    {
      "name":"Content",
      "type":"string",
      "value":"Item_Coin_Bronze"
    }
  ],
  "rotation":0,
  "type":"",
  "visible":true,
  "width":128,
  "x":1280,
  "y":1024
}
```

In this case, the content property, which is a string, tells the engine that the box contains a bronze coin. When the box is destroyed, the player will collect a bronze coin.

3 User Stories (100 points)

1. As a user, I want to login using my SWITCH credentials, so that I can access Blockhop	10 pts
Once the Blockhop website is loaded, users can login via SWITCH Edu-Id credentials. ¹ If they are not logged in, the page will show a "login" button that will redirect the user to the eduid login page. The user needs to be a USI user.	
2. As a player, after logging in, I can get the list of available levels.	10 pts
A logged-in user can see the list of the available levels published by other creators (including the ones created by themselves). The levels are shown with a small thumbnail of the level map, a title, and a description. They must be sorted by popularity, i.e., the number of times they have been played in a certain period, or by clear rate.	
3. As a user, I can access my Blockhop personal page	5 pts
In a dedicated page, users can see their profile, where they can inspect information about how many levels they have played and successfully completed. Furthermore, one can see all the levels they have created, both published or not.	
4. As a player, I can access and play an available level	5 pts
As a player, I can enter a level from the main page and attempt it. A level can be attempted an infinite number of times, until the player completes it.	
5. As a creator, I can create a new level.	5 pts
As a creator, from my personal page, I can create a new level - either from scratch or starting from another level they have created previously and modify that.	
6. As a creator, I can edit the properties of an unpublished level.	5 pts
A level can be modified only when it has not been published. After that, it cannot be modified anymore. The property editor allows to modify the title, the description, and the clear condition of a level.	
7. As a creator, I can edit the world layer of an unpublished level.	20 pts
The level map editor allows first to edit the world layer of an unpublished level. Only ground tiles can be inserted in this layer.	
8. As a creator, I can edit the object layer of an unpublished level.	20 pts
As a creator, I can add objects in tiles where there is no ground layer. Only one object can be inserted in a given tile.	
9. As a creator, I can edit the properties of an object	5 pts
In the object layer editor, as a creator, I can customize the properties of some items that have them. For example, for boxes, there can be some content. Only coins are allowed to be content of a box.	

¹ the same ones that you use for iCorsi.

10. As a creator, I can publish a level.	10 pts
As a creator, I can publish a level. Before the level can be published, the creator needs to complete it from scratch. Only when the level has been completed, and not further edited, then it can be published. From that moment on, the level cannot be edited anymore.	
11. As a creator, I can unpublish a level.	5 pts
A level can be unpublished by the creator. In that case, it stays hidden and it is not visible anymore.	

4 Development Requirements

A good implementation of the system requires a good comprehension of the requirements, a good object-oriented analysis step (for the conceptual model of your system), and a good design. You can use any of the methodologies that we listed during the lectures, including:

- constructing a good domain model of the system, including a phase where you do OO analysis on this document to understand which classes should be there (*e.g.*, Level, Object, Enemy etc.)
- the use of UML diagrams - *e.g.*, , to depict scenarios, for example for what the frontend (our component that will have the role of the view) should do when a new level needs to be published. Obviously, you need to wait after the frontend and UML lectures.

This will help you to communicate with your team members, but also to communicate with us in case of need. Some requirements are imprecise, are probably ambiguous, and there may be many different ways to interpret them. This does not mean that all interpretations are valid: go back to your stakeholders (*i.e.*, us) if you are not sure what something means or entails.

All documentation should be on GitLab, *i.e.*, mostly on issues and merge requests. Later, if you want, you can have a dedicated project for documentation.

Given the fact that we still did not do the frontend lab lecture, you should start working on the backend. We will set up a base project which includes the code of Lab 02 and a basic CI pipeline. You should ensure that all your team members have seen the first two labs, and that they understand the concept of the lectures that involve coding.

During the first days of the project, you should mainly go over the document and analyze it. As a first step, you should open a Milestone for the first batch.

Then, other possible initial tasks include understanding in which order you can implement the user stories, if there is any dependency. Start opening issues that track the completion of each user story. Write in the issue description the user story itself and any possible other element of the document that involves them. You can assign a pair of people to one/two user stories. Their responsibility would be to derive dependencies and aspects of Section 1 that are related to the user stories. Then, as a team, you can go through them again and starting doing some OO analysis for the model (as in DDD), and start sketching the code for it (some user stories, though, do not involve the model).

Do not try to work on a entire user story in one single issue with its own merge request. It's better to create sub-issues for specific tasks related to one or more user stories: For example, modeling an exercise or a question would likely be needed for more user stories. Such smaller issues can have smaller merge requests that are easier to integrate.

When working on the domain model, you should be using what you know from the lectures on specifications, design, OOA, and DDD, and from the previous courses. A pair of team members can start investigating the logging in user story - this is nontrivial, but making it work is quite fun. Please make sure everybody understands what are the implications of this user story when completed - though its completion will require work on the frontend as well. Try not to "copy" it from previous years: some configurations can be changed and thus it may not work as it was anymore. Furthermore, always pay a LOT of attention to the suggestions of an AI assistant. It may suggest you overly complex designs, or just produce messy code, that your team members will hate. You always have the full responsibility of the code you produce. If your team members complain that your code does not meet what was explained in this course, they can rise the issue with the instructor and

you will get significant penalties, since your contributions will be discarded and will not count to your personal grade in the project.

Do not try to finish the model before working on services and controllers. Try to start from a basic skeleton of classes in the domain model, even with minimal logic, and then try to understand which services, repositories, and controllers you will need. Focus testing on the domain model only - do not try to test services, controllers, or any springboot component before the Testing (2) lecture.

Concerning some data types - try to **not reinvent the wheel** (in particular for dates and times).

5 Technical Details on Architectural Requirements

Logging in with Switch Edu-ID

For this task, it is extremely important that you do not follow random tutorials online, but that you follow these instructions. If you get stuck, you should ask for help from the instructor. It is important that the people assigned to this issue are skilled, and before integrating the login changes you explain to the other team members what are the implications - after these changes, it will be very difficult to test the requests to the backend with postman.

Logging in with Switch Edu-Id is supported through a series of protocols, mainly OAuth 2 and OpenID. These protocols are very complicated, and you are not supposed to manipulate them explicitly or to know many details about it. Fortunately, the Spring Security library that we added as a dependency in the second lab already implements the functionality to support logging in through these protocols.

First of all, you need to add the following dependencies to your project:

```
implementation 'org.springframework.boot:spring-boot-starter-oauth2-client'
implementation 'org.springframework.boot:spring-boot-starter-oauth2-resource-server'
```

Then, add the following configuration to your application.properties file that is present in the main folder:

```
spring.security.oauth2.client.registration.switch-edu-id.client-id=test-client
spring.security.oauth2.client.registration.switch-edu-id.authorization-grant-type=authorization_code
spring.security.oauth2.client.registration.switch-edu-id.scope=openid
spring.security.oauth2.client.provider.switch-edu-id.issuer-uri=https://auth.dev.si.usi.ch/auth/realms/simurbs
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://auth.dev.si.usi.ch/auth/realms/simurbs
```

```
logging.level.org.springframework.security=DEBUG
```

The last one is particularly useful for debugging.

Trying the Log in process

In the first part of this batch, you are not supposed to construct a frontend for the application, but this would make the testing of the login with Gitlab a little hard to do with just postman.

First of all, try to require authentication for a specific route in the controller. You should modify the SecurityConfiguration class in your project. In the actual status, it permits all calls. You should change it by requiring - for some routes - to be authenticated with OAuth2. It is simple and it requires only to modify the SecurityConfiguration class, and in particular the filterChain method. If you want, you may have a look to some tutorials online, but please try not to get lost. If you're writing too much code for this, you are probably making some mistake. We can help.

When things work well, when you try to get from the browser a route that requires authentication, you should be redirected to the Switch Edu-Id page to log in. After a successful log in, you will see the right output for the route.

Retrieve user data after authentication

The first route that you could authenticate is for example one that will return some user data as it is coming from Switch Edu-Id.

It will look like this:

```
record SwitchEduIdUser(String fullName) {}

@GetMapping(path = "/me")
public ResponseEntity<SwitchEduIdUser> index(OAuth2AuthenticationToken authentication) {
    String fullName = authentication.getPrincipal().getAttribute("name");
    return ResponseEntity.ok(new SwitchEduIdUser(fullName));
}
```

You can imagine the authentication variable to represent the user data that you get after a successful login. This controller method will return the full name of the user. If you have properly authenticated this route (under the UserController), you will see your username.

Final Remarks

The user story 1 implies that your project will not manage authentication, and passwords, directly. This does not mean that you shouldn't have users in your domain model though. You need a way to 'map' a user from Switch Edu-Id (as you get from the OAuth2AuthenticationToken) to a user in the application you are developing. Switch Edu-Id users already have an identifier that you can essentially reuse - but again, this is something that you need to figure out.